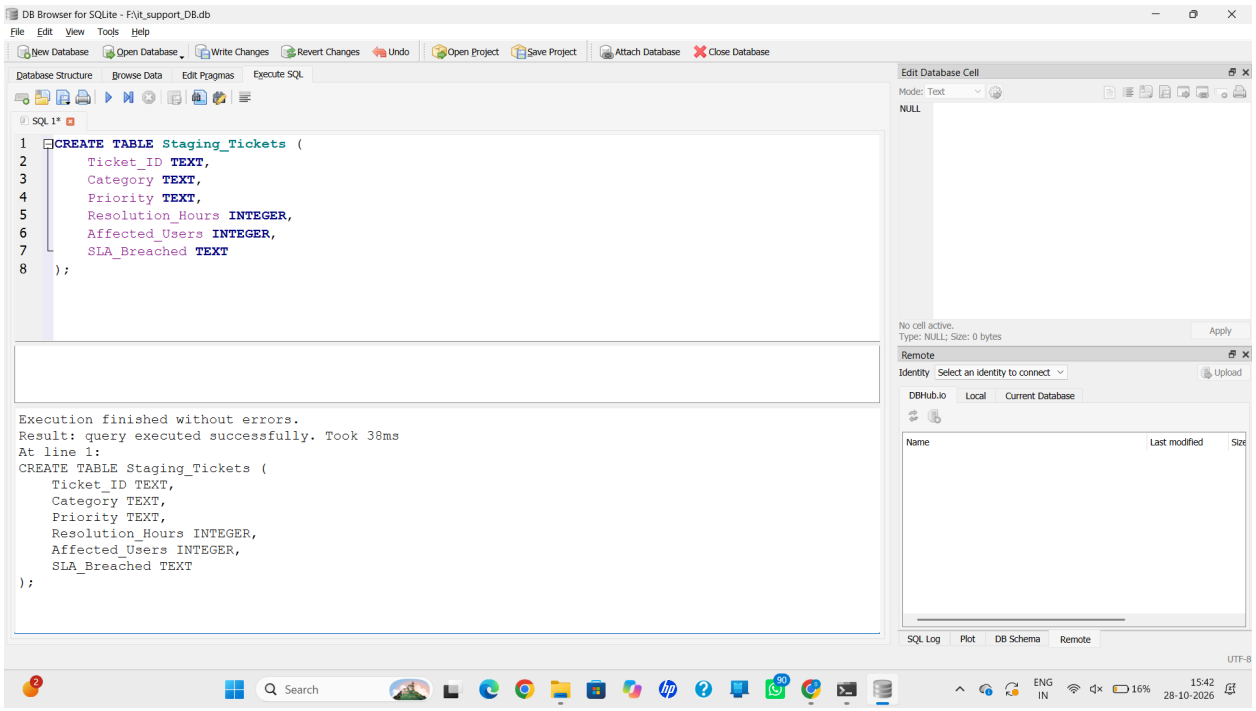
Dataset1 Documentation

Open DB Browser for SQLite

Run this SQL command: Create a staging table

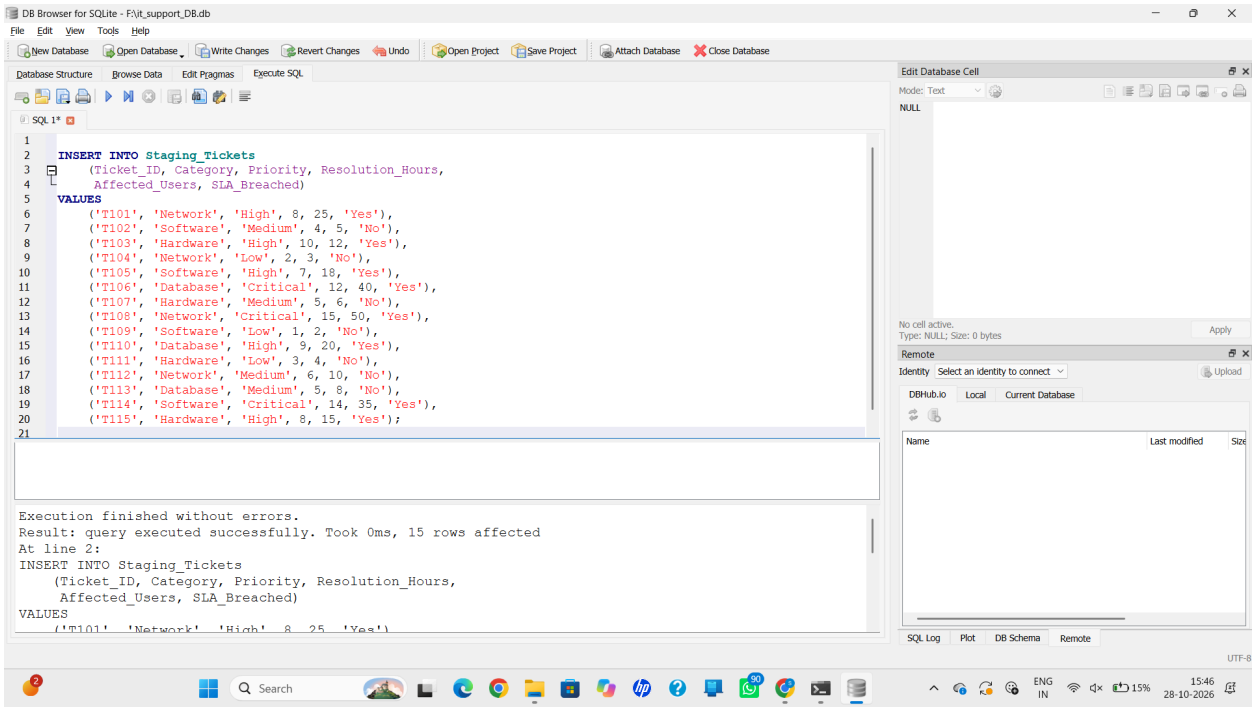
```
CREATE TABLE Staging_Tickets (
    Ticket_ID TEXT,
    Category TEXT,
    Priority TEXT,
    Resolution_Hours INTEGER,
    Affected_Users INTEGER,
    SLA_Breached TEXT
);
```



Inserting all CSV data :

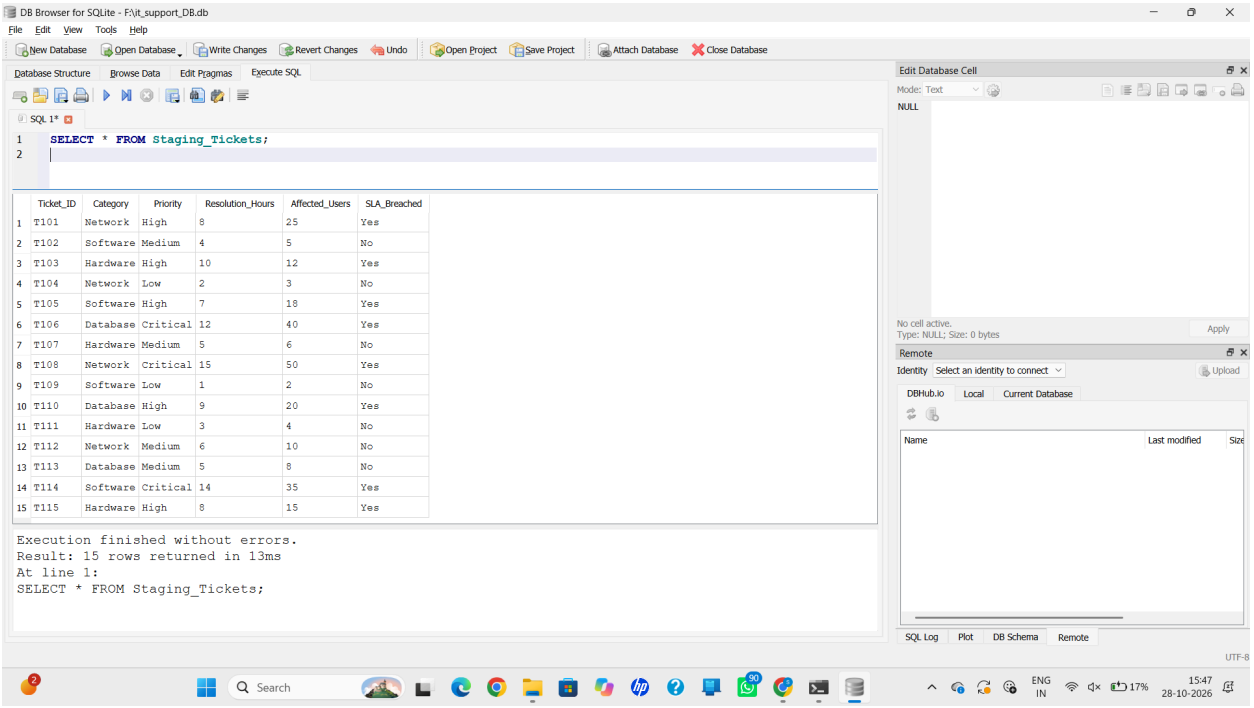
```
INSERT INTO Staging_Tickets
    (Ticket_ID, Category, Priority, Resolution_Hours,
     Affected_Users, SLA_Breached)
VALUES
    ('T101', 'Network', 'High', 8, 25, 'Yes'),
    ('T102', 'Software', 'Medium', 4, 5, 'No'),
    ('T103', 'Hardware', 'High', 10, 12, 'Yes'),
```

('T104', 'Network', 'Low', 2, 3, 'No'),
('T105', 'Software', 'High', 7, 18, 'Yes'),
('T106', 'Database', 'Critical', 12, 40, 'Yes'),
('T107', 'Hardware', 'Medium', 5, 6, 'No'),
('T108', 'Network', 'Critical', 15, 50, 'Yes'),
('T109', 'Software', 'Low', 1, 2, 'No'),
('T110', 'Database', 'High', 9, 20, 'Yes'),
('T111', 'Hardware', 'Low', 3, 4, 'No'),
('T112', 'Network', 'Medium', 6, 10, 'No'),
('T113', 'Database', 'Medium', 5, 8, 'No'),
('T114', 'Software', 'Critical', 14, 35, 'Yes'),
('T115', 'Hardware', 'High', 8, 15, 'Yes');



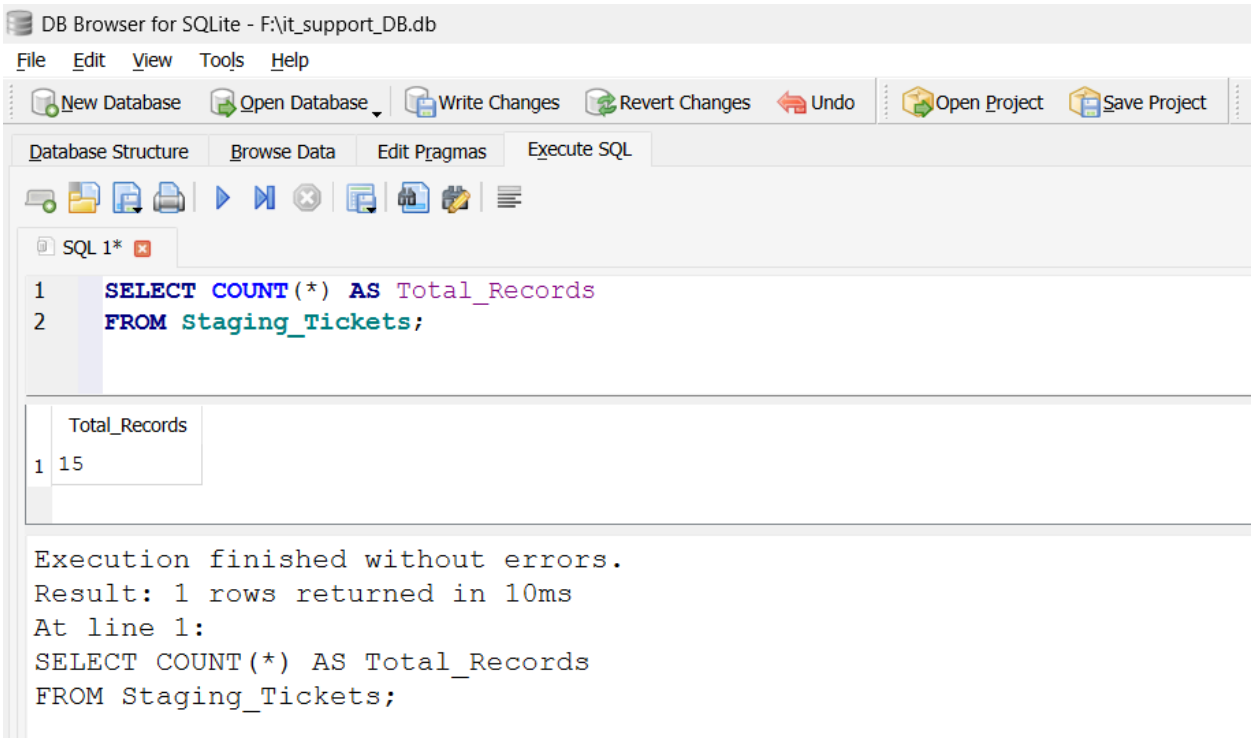
Verifing our data :

SELECT * FROM Staging_Tickets;



No of records inserted:

SELECT COUNT(*) AS Total_Records FROM Staging_Tickets;



Create the Categories table :

```
CREATE TABLE Categories (
   category_id INTEGER PRIMARY KEY AUTOINCREMENT,
   category_name TEXT NOT NULL UNIQUE
);
```

Create the Priorities table :

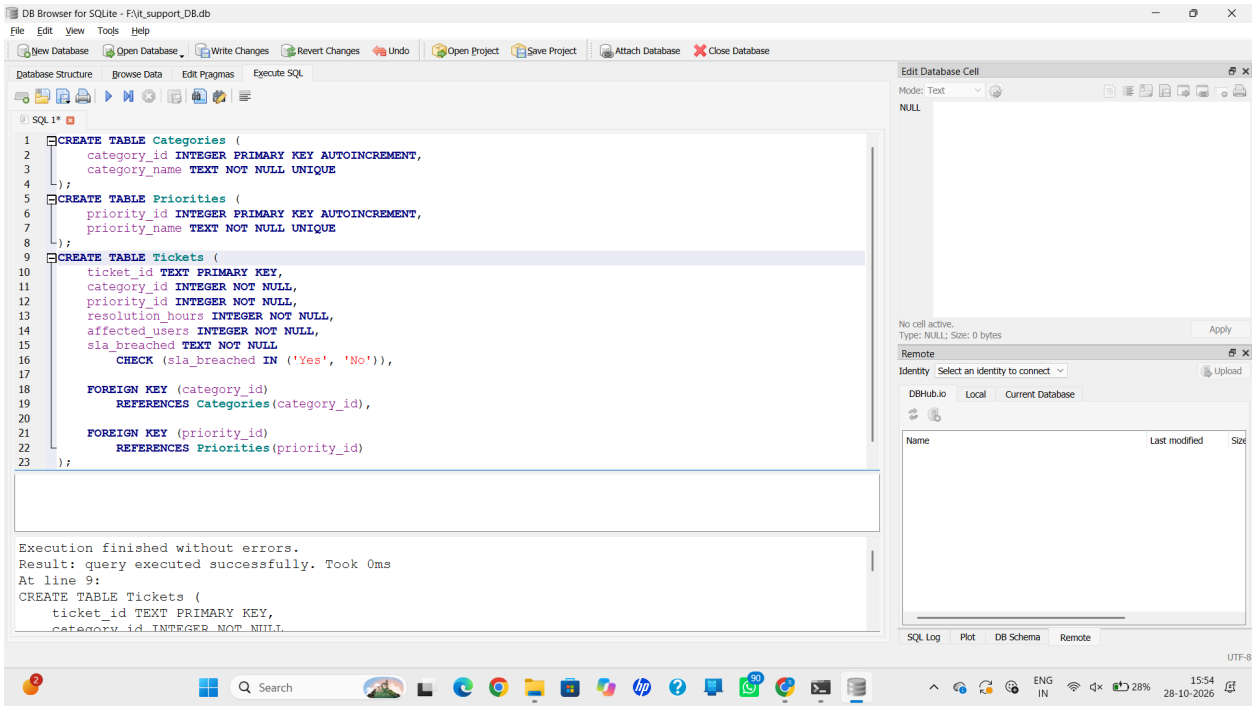
```
CREATE TABLE Priorities (
   priority_id INTEGER PRIMARY KEY AUTOINCREMENT,
   priority_name TEXT NOT NULL UNIQUE
);
```

Create the Tickets table :

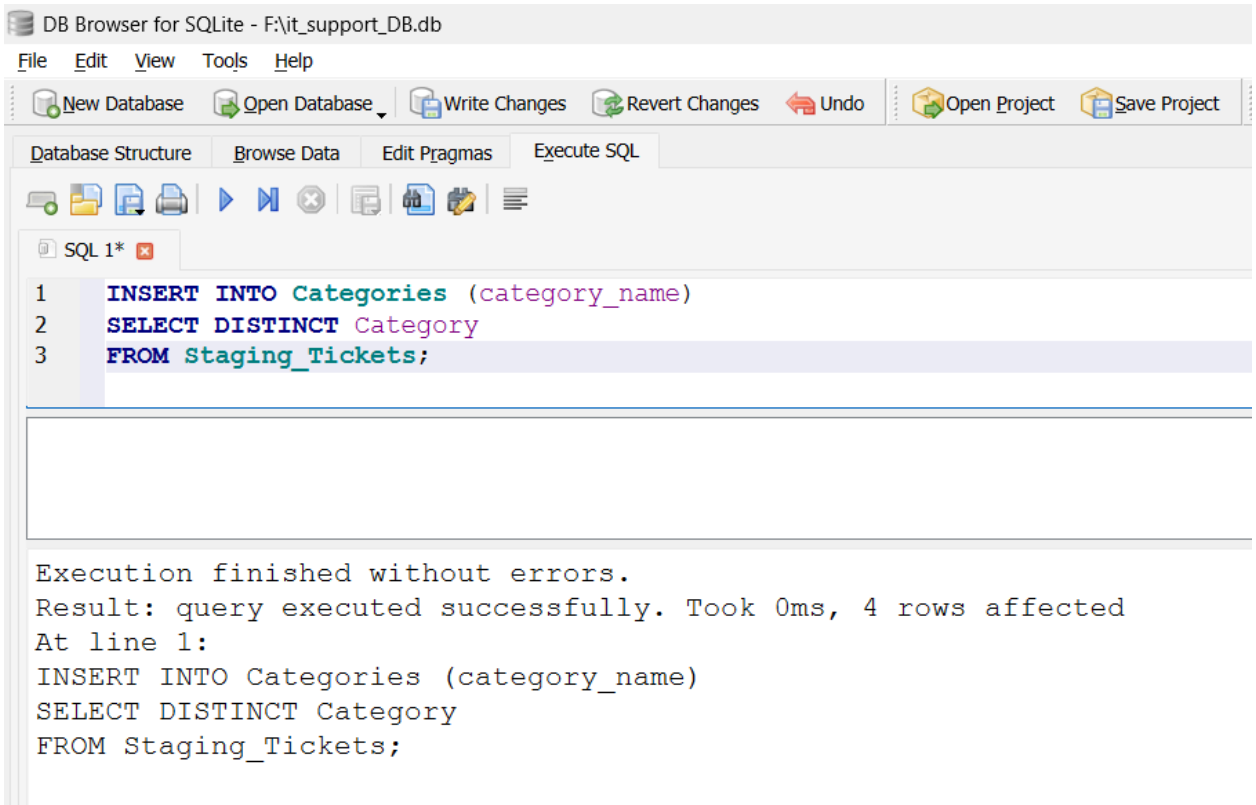
```
CREATE TABLE Tickets (
   ticket_id TEXT PRIMARY KEY,
   category_id INTEGER NOT NULL,
   priority_id INTEGER NOT NULL,
   resolution_hours INTEGER NOT NULL,
   affected_users INTEGER NOT NULL,
   sla_breached TEXT NOT NULL
      CHECK (sla_breached IN ('Yes', 'No')),

   FOREIGN KEY (category_id)
      REFERENCES Categories(category_id),

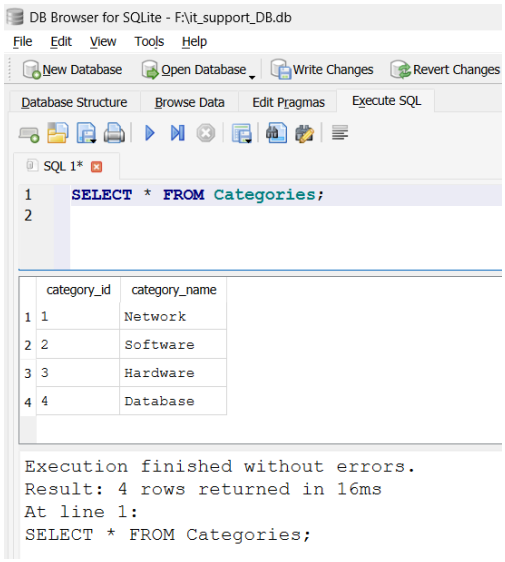
   FOREIGN KEY (priority_id)
      REFERENCES Priorities(priority_id)
);
```



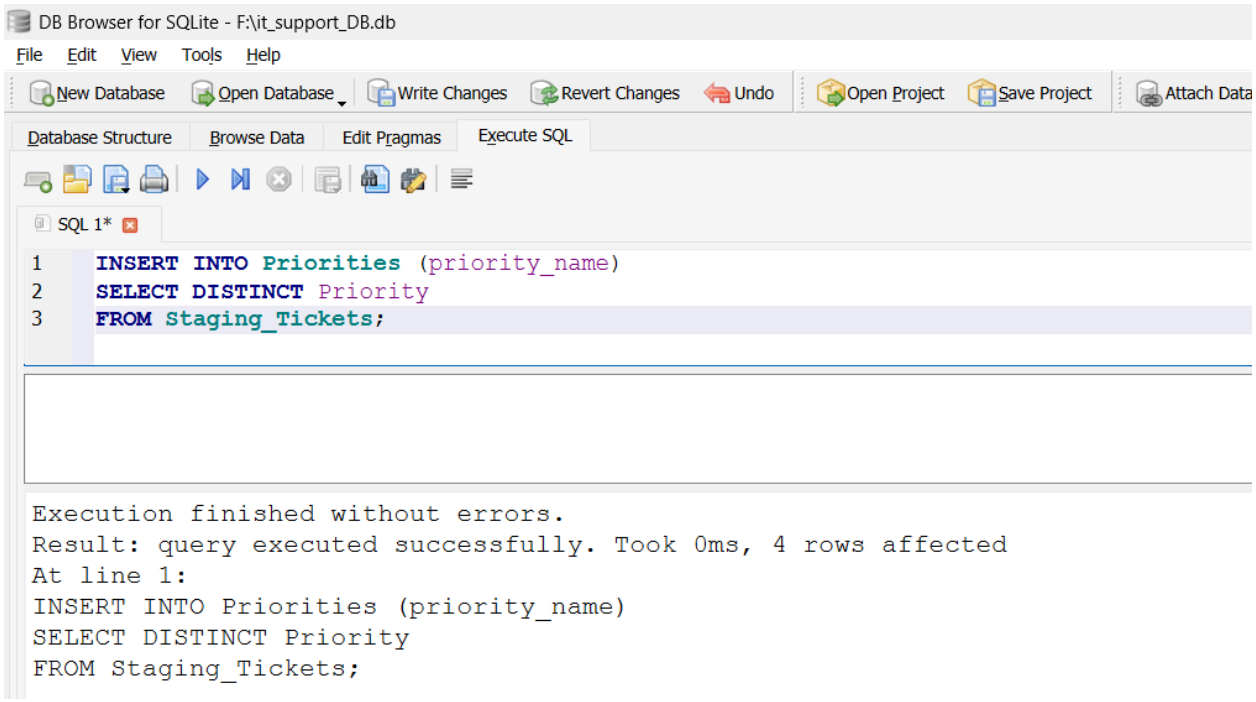
INSERT INTO Categories (category_name)
SELECT DISTINCT Category
FROM Staging_Tickets;



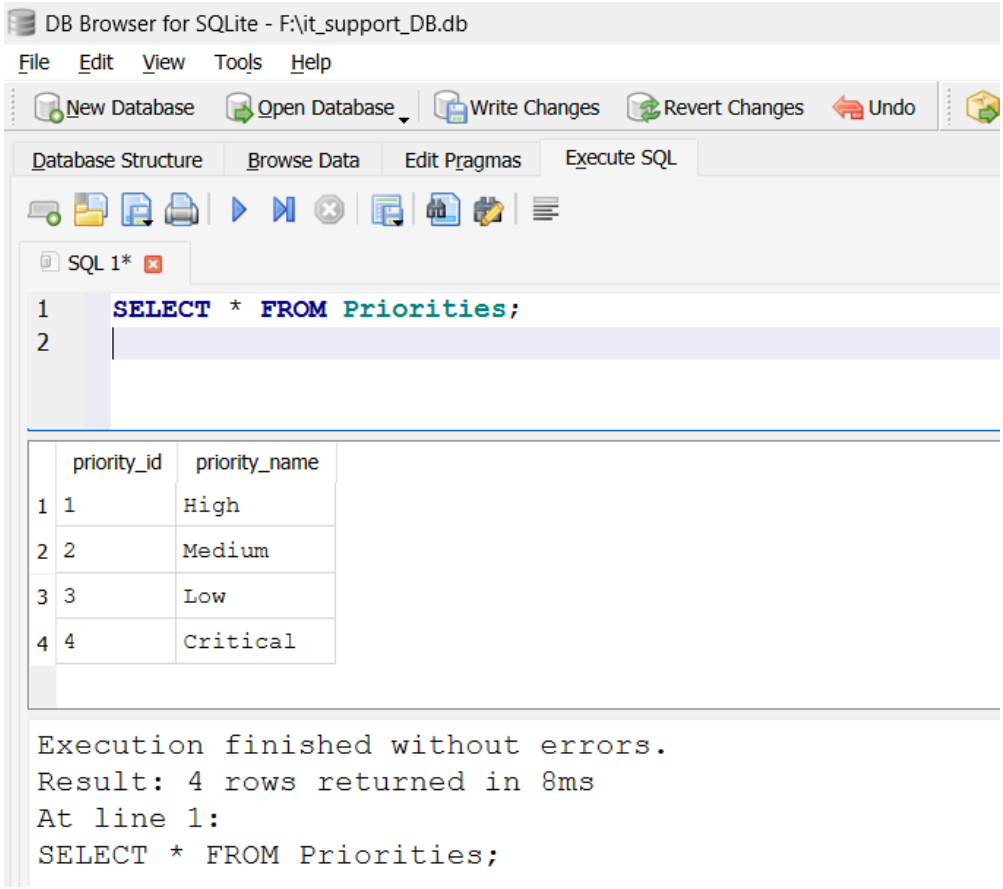
SELECT * FROM Categories;



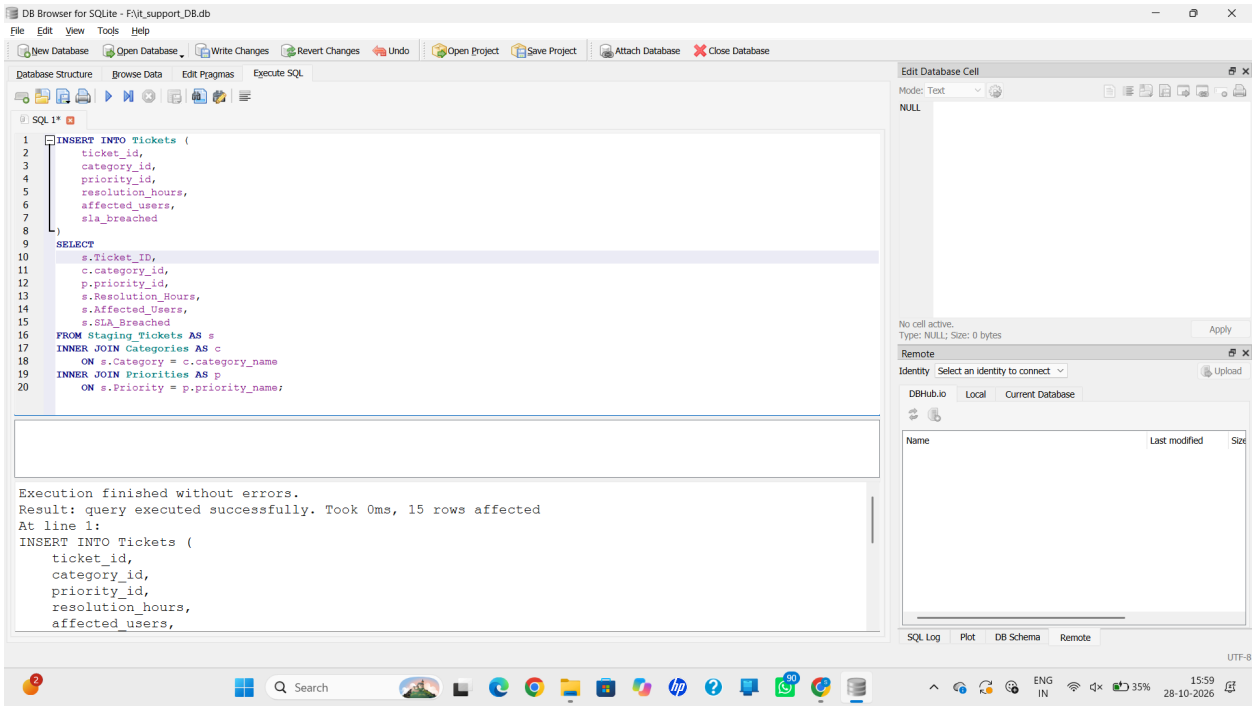
INSERT INTO Priorities (priority_name)
SELECT DISTINCT Priority
FROM Staging_Tickets;



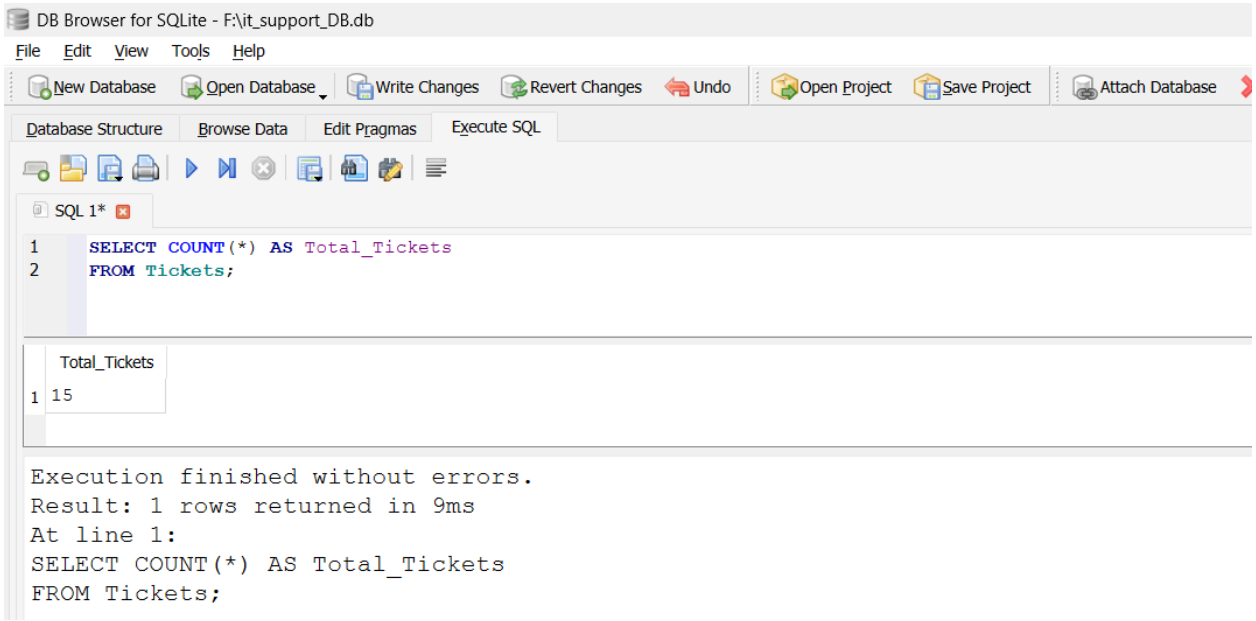
SELECT * FROM Priorities;




```
INSERT INTO Tickets (
    ticket_id,
    category_id,
    priority_id,
    resolution_hours,
    affected_users,
    sla_breached
)
SELECT
    s.Ticket_ID,
    c.category_id,
    p.priority_id,
    s.Resolution_Hours,
    s.Affected_Users,
    s.SLA_Breached
FROM Staging_Tickets AS s
INNER JOIN Categories AS c
    ON s.Category = c.category_name
INNER JOIN Priorities AS p
    ON s.Priority = p.priority_name;
```

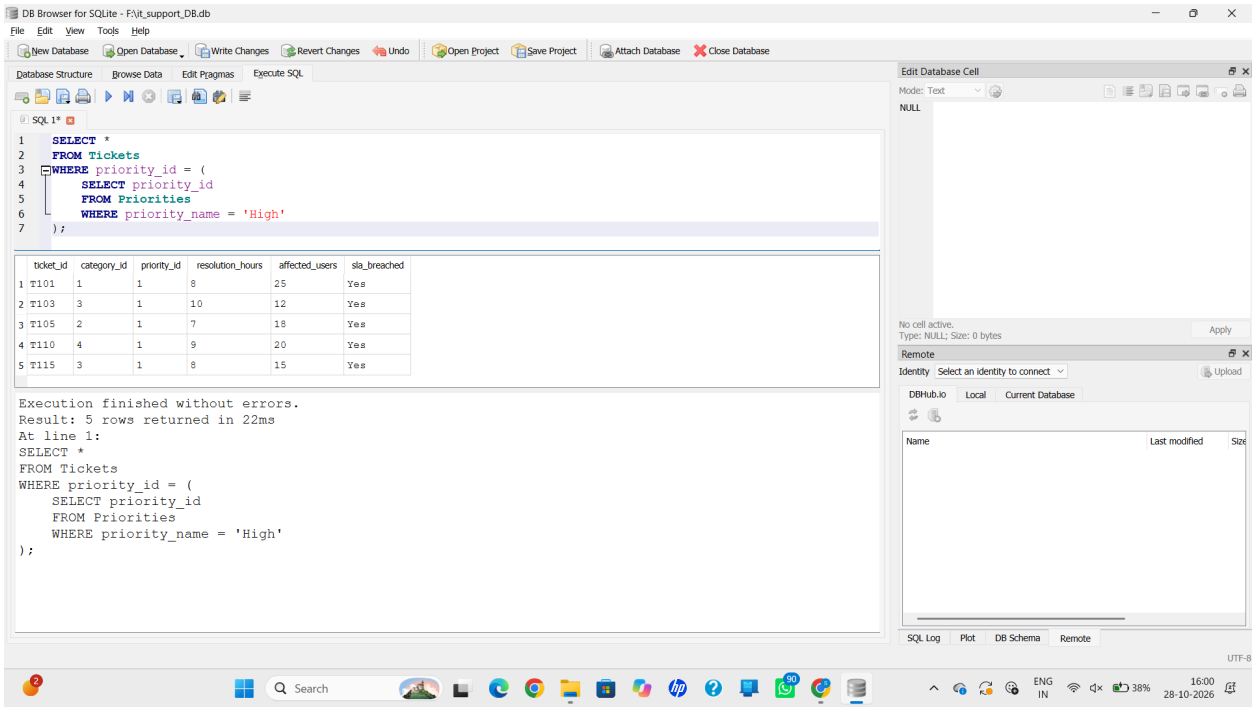


SELECT COUNT(*) AS Total_Tickets
FROM Tickets;



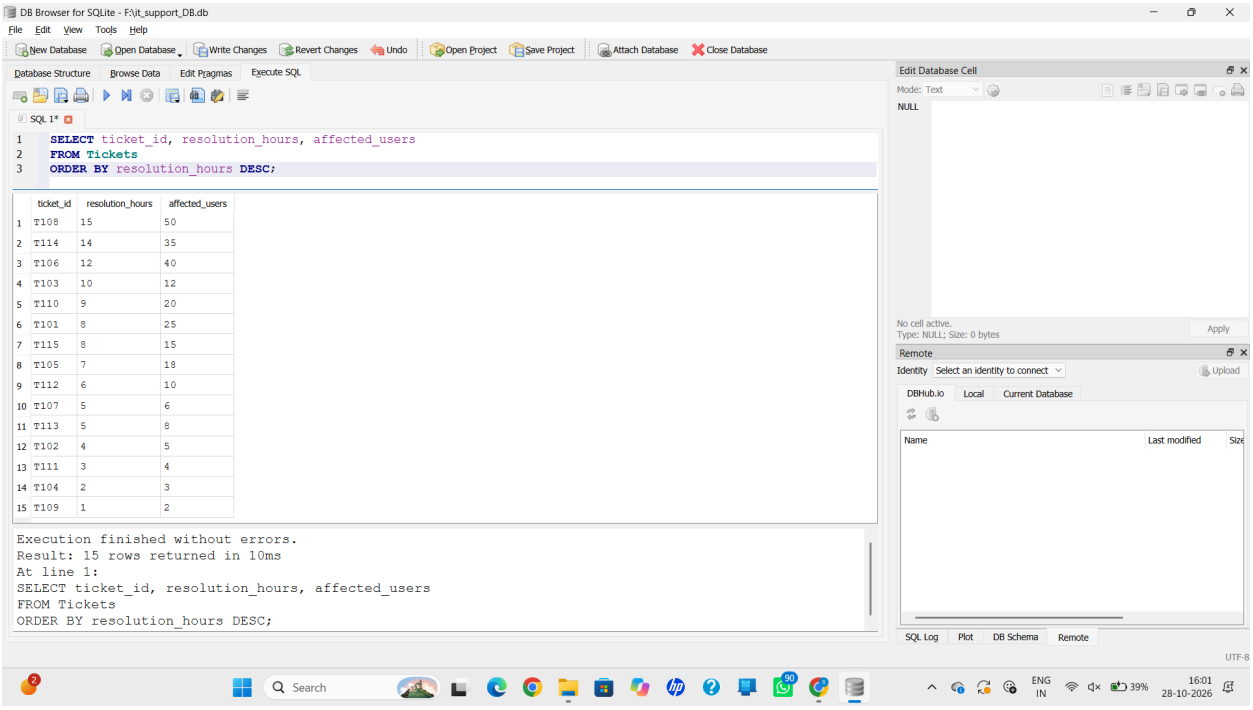
Filter high-priority tickets :

```
SELECT *
FROM Tickets
WHERE priority_id = (
    SELECT priority_id
    FROM Priorities
    WHERE priority_name = 'High'
);
```



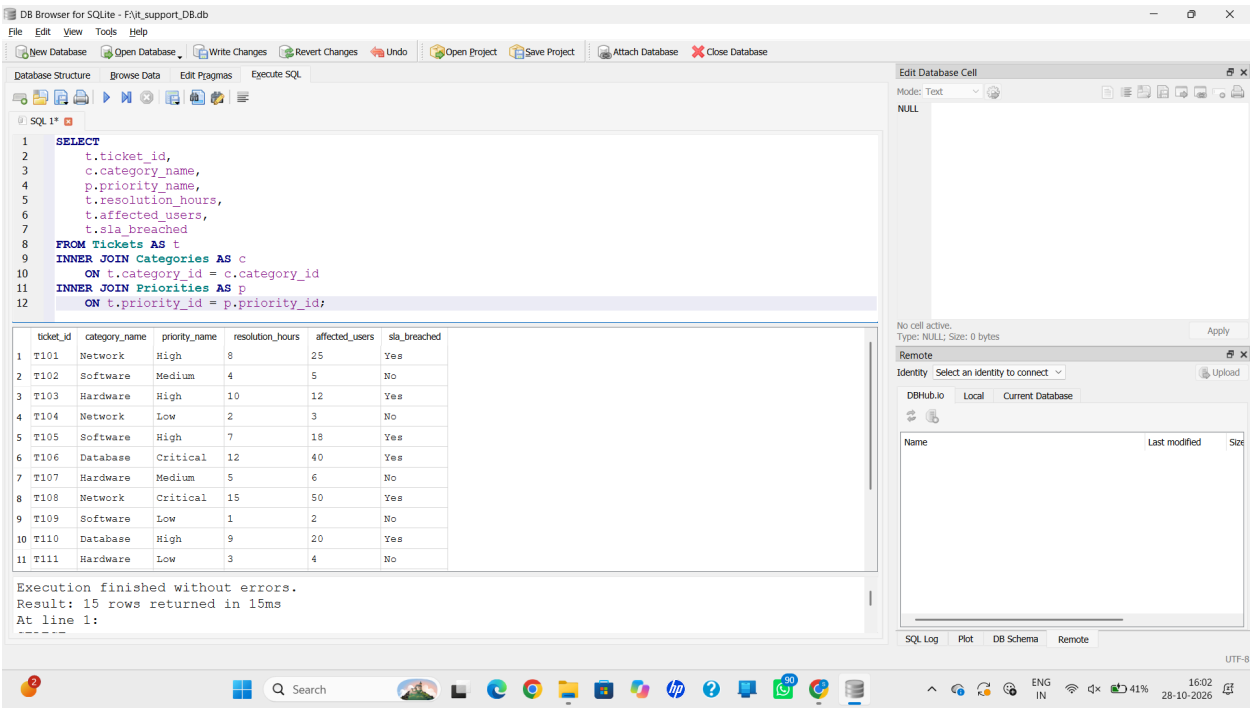
Sort tickets by resolution time

```
SELECT ticket_id, resolution_hours, affected_users
FROM Tickets
ORDER BY resolution_hours DESC;
```



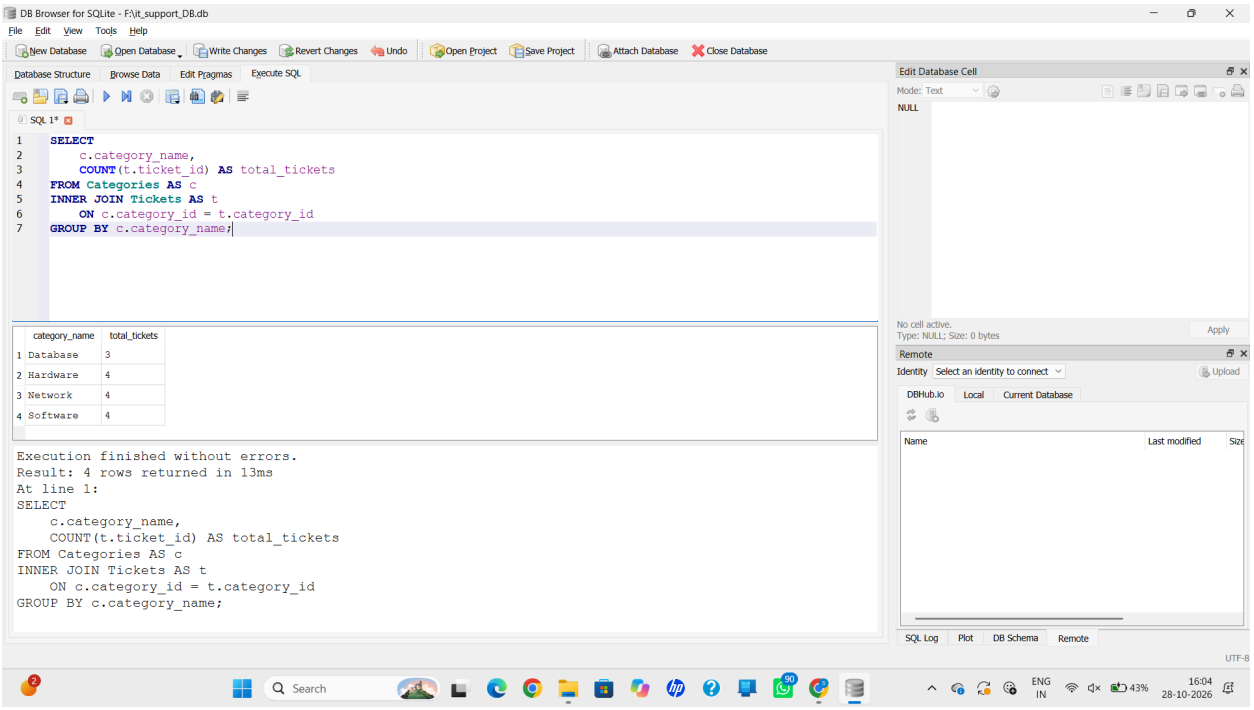
Using INNER JOIN :Display each ticket with its category and priority names instead of numeric IDs.

```
SELECT
    t.ticket_id,
    c.category_name,
    p.priority_name,
    t.resolution_hours,
    t.affected_users,
    t.sla_breached
FROM Tickets AS t
INNER JOIN Categories AS c
    ON t.category_id = c.category_id
INNER JOIN Priorities AS p
    ON t.priority_id = p.priority_id;
```



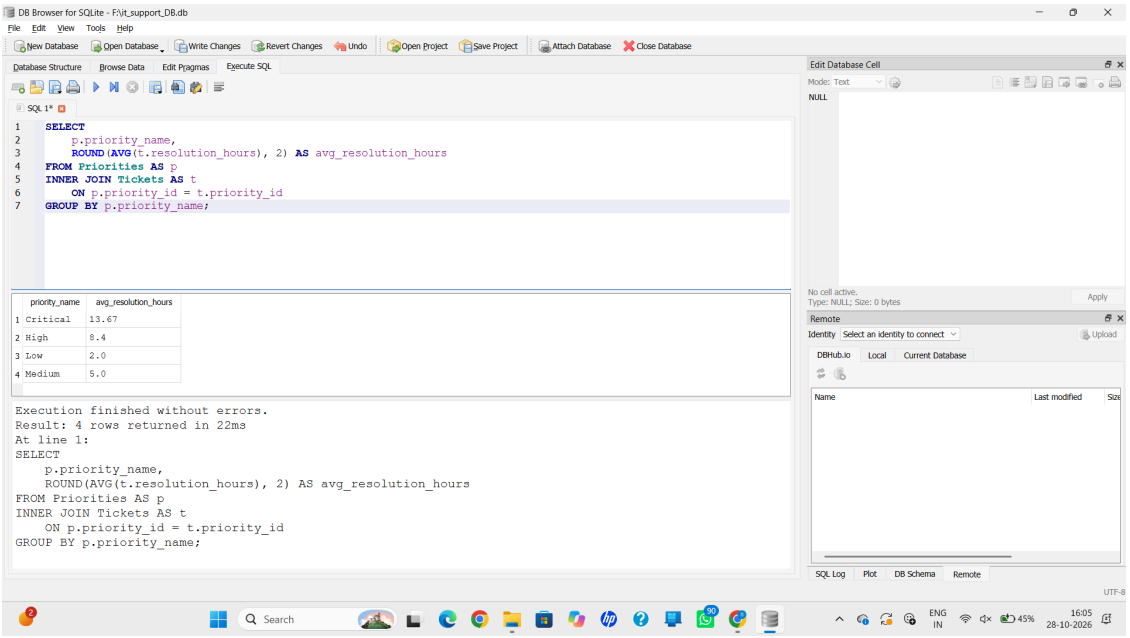
GROUP BY and COUNT :Count the number of tickets in each category.

```
SELECT
    c.category_name,
    COUNT(t.ticket_id) AS total_tickets
FROM Categories AS c
INNER JOIN Tickets AS t
    ON c.category_id = t.category_id
GROUP BY c.category_name;
```



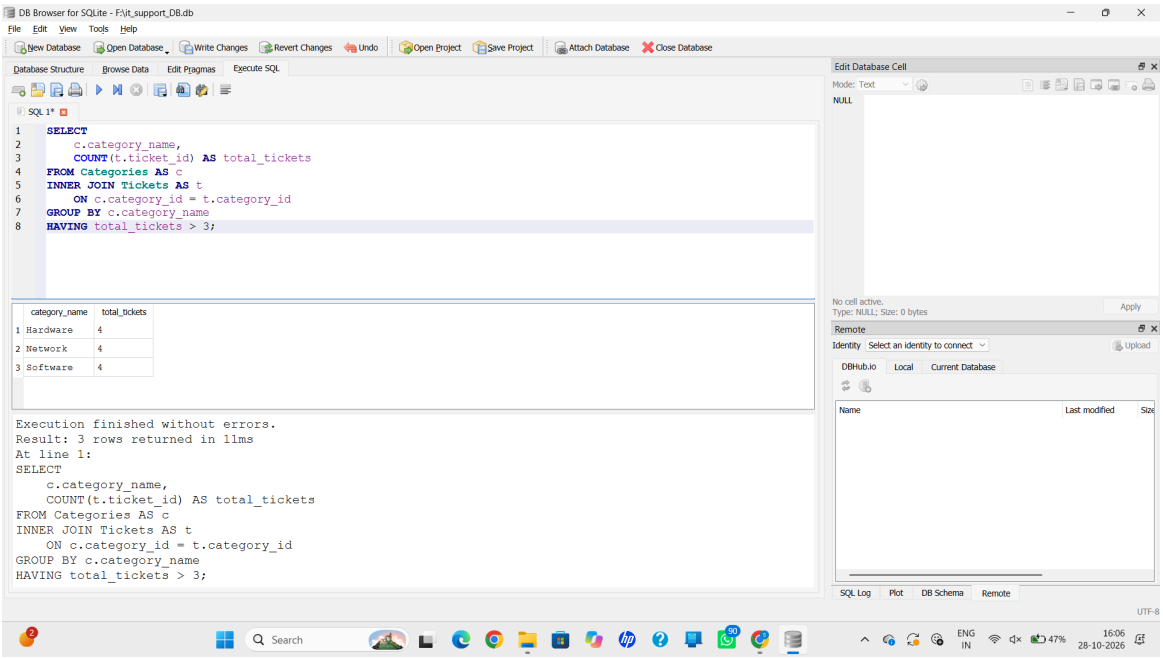
GROUP BY and AVG :Calculate the average resolution time for each priority.

```
SELECT
    p.priority_name,
    ROUND(AVG(t.resolution_hours), 2) AS avg_resolution_hours
FROM Priorities AS p
INNER JOIN Tickets AS t
    ON p.priority_id = t.priority_id
GROUP BY p.priority_name;
```



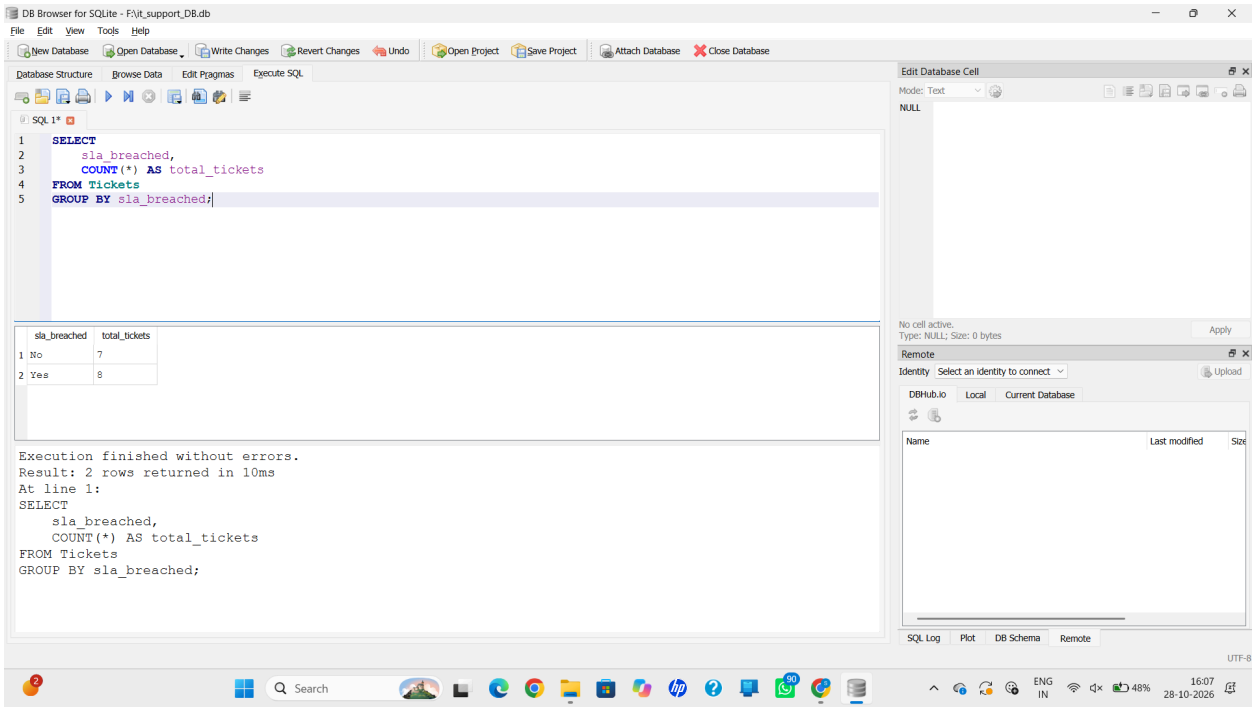
GROUP BY and HAVING :Find categories with more than three tickets.

```
SELECT
    c.category_name,
    COUNT(t.ticket_id) AS total_tickets
FROM Categories AS c
INNER JOIN Tickets AS t
    ON c.category_id = t.category_id
GROUP BY c.category_name
HAVING total_tickets > 3;
```



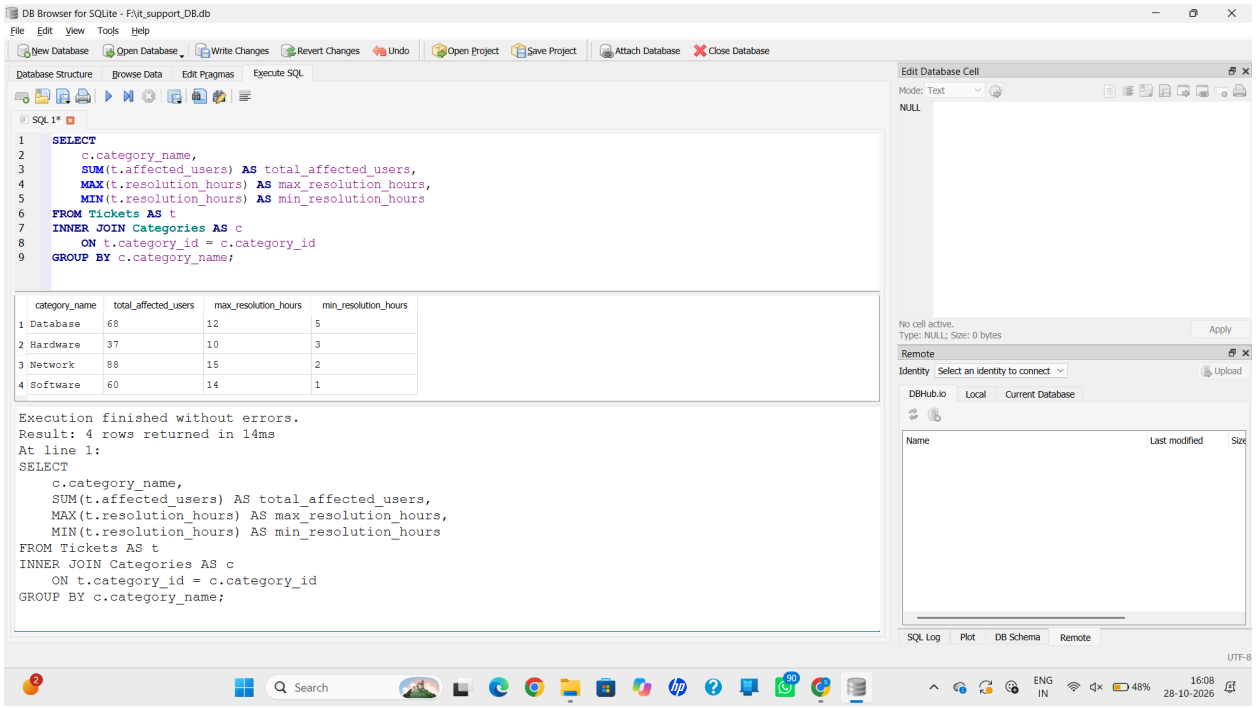
Find SLA breaches :Count tickets that breached the SLA and tickets that did not.

```
SELECT
    sla_breached,
    COUNT(*) AS total_tickets
FROM Tickets
GROUP BY sla_breached;
```



SUM, MAX and MIN :Summarize affected users and resolution times by category.

```
SELECT
    c.category_name,
    SUM(t.affected_users) AS total_affected_users,
    MAX(t.resolution_hours) AS max_resolution_hours,
    MIN(t.resolution_hours) AS min_resolution_hours
FROM Tickets AS t
INNER JOIN Categories AS c
    ON t.category_id = c.category_id
GROUP BY c.category_name;
```



Find the longest-running ticket

```
SELECT ticket_id, resolution_hours, affected_users
FROM Tickets
WHERE resolution_hours = (
    SELECT MAX(resolution_hours)
    FROM Tickets
);
```

